Séance 05 - Fonctions et décomposition

Table of Contents

1. Situation professionnelle . . . 1
2. Objectifs . . . 1
3. Environnement et réactivation . . . 2
4. Nouveau réflexe - le typage strict . . . 2
5. Activité 1 - Déclarer, appeler, retourner . . . 2
6. Activité 2 - Plusieurs paramètres . . . 3
7. Activité 3 - Retourner un booléen . . . 3
   7.1. Premier commit . . . 4
8. Activité 4 - Transformer un tableau . . . 4
9. Activité 5 - Une fonction, une responsabilité . . . 4
10. Activité 6 - Séparer les fichiers . . . 5
11. Activité 7 - Tests simples . . . 5
12. Diagnostic . . . 6
13. Exercice autonome . . . 6
14. Exercices bonus . . . 6
   14.1. Exercice bonus 1 [*] - Un paramètre par défaut . . . 6
   14.2. Exercice bonus 2 [**] - Une fonction qui borne une valeur . . . 7
   14.3. Exercice bonus 3 [*] - Défi : un mini-outil de test . . . 7
15. Synthèse et Git . . . 7

1. Situation professionnelle

Les calculs et classifications sont maintenant répétés dans plusieurs scripts. Vous allez isoler ces règles dans des fonctions réutilisables et testables.

2. Objectifs

- déclarer et appeler une fonction ;
- distinguer paramètre, argument et valeur retournée ;
- utiliser des types simples dans une signature ;
- préférer `return` à un affichage dans une fonction de calcul ;
- décomposer un problème en fonctions courtes ;
- séparer les fonctions de leur programme d'utilisation avec `require_once`.

3. Environnement et réactivation

```
git status
git pull
docker compose up -d
```



Ouvrez le dossier du TP et tapez `docker compose up -d`. Vos fichiers `.php` (y compris le dossier `lib/`) vont **dans le dossier du TP** ; ils s'ouvrent sur `http://localhost:8181/nom-du-fichier.php`. Guide pas à pas : `cours/annexe-docker-php.adoc`.

1. Pourquoi répéter le même code est-il risqué ?
2. Quelle différence existe entre calculer une valeur et l'afficher ?
3. Quel type PHP représente vrai ou faux ?

4. Nouveau réflexe - le typage strict

À partir de cette séance, les fonctions reçoivent des **types** (`float`, `int`, `bool`, `string`, `array`). Pour que ces types soient réellement respectés, chaque fichier de la séance commence désormais par :

```
<?php

declare(strict_types=1);
```

Sans `strict_types`, PHP **convertit silencieusement** les arguments. Avec, il refuse un type inattendu et lève une erreur immédiate — ce qui aide à repérer les bugs tôt :

Appel	**Sans** `strict_types`	**Avec** `strict_types=1`
`estPair("12")` (chaîne au lieu d'un `int`)	`true` (la chaîne est convertie)	`TypeError` (type refusé)
`calculerCoutAnnuel(25)` (un `int` pour un `float`)	`300.0`	`300.0` (un `int` est accepté là où un `float` est attendu)



`declare(strict_types=1);` doit être la **première instruction** du fichier, juste après `<?php`. Il n'agit que sur les appels de fonctions situés dans ce fichier.

5. Activité 1 - Déclarer, appeler, retourner

Créez `fonctions.php` :

```
<?php
```

```php
declare(strict_types=1);

function calculerCoutAnnuel(float $coutMensuel): float
{
    return $coutMensuel * 12;
}

$coutServeur = calculerCoutAnnuel(24.90);

echo "Coût annuel : " . $coutServeur . " euros";
```

Vocabulaire :

- `calculerCoutAnnuel` : nom ;
- `$coutMensuel` : paramètre ;
- `24.90` : argument ;
- `float` après les parenthèses : type retourné ;
- `return` : renvoie le résultat à l'appelant.

Appelez la fonction avec trois coûts différents.

6. Activité 2 - Plusieurs paramètres

```php
function calculerTauxOccupation(float $espaceUtilise, float $capacite): float
{
    return $espaceUtilise / $capacite * 100;
}
```

Testez `calculerTauxOccupation(360, 500)`.

Ajoutez une validation : si la capacité est inférieure ou égale à zéro, retournez `0`.

QUESTION: Pourquoi les paramètres sont-ils préférables à des variables écrites directement dans la fonction ?

7. Activité 3 - Retourner un booléen

```php
function estChargeCritique(float $chargeCpu): bool
{
    return $chargeCpu >= 90;
}
```

Utilisez le résultat dans un `if`. Testez `89.9`, `90` et `100`.

Écrivez `estPair(int $nombre): bool` avec l'opérateur `%`.

7.1. Premier commit

```
git add 05-fonctions/fonctions.php
git commit -m "Ajoute les premières fonctions PHP"
git push
```

8. Activité 4 - Transformer un tableau

```
function ajouterPrefixe(array $nomsServeurs, string $prefixe): array
{
    $nomsComplets = [];

    foreach ($nomsServeurs as $nomServeur) {
        $nomsComplets[] = $prefixe . $nomServeur;
    }

    return $nomsComplets;
}
```

Testez avec `["web-01", "bdd-01"]` et le préfixe `"srv-"`.

QUESTION: Pourquoi crée-t-on un nouveau tableau ?

9. Activité 5 - Une fonction, une responsabilité

Décomposez un rapport de supervision en trois fonctions :

```
function determinerEtat(float $chargeCpu): string
{
    if ($chargeCpu >= 90) {
        return "CRITIQUE";
    }

    if ($chargeCpu >= 70) {
        return "SURVEILLANCE";
    }

    return "NORMAL";
}

function calculerMoyenne(array $valeurs): float
```

```
{
    if (count($valeurs) === 0) {
        return 0;
    }

    return array_sum($valeurs) / count($valeurs);
}
```

Ajoutez `construireMessage(string $nomServeur, string $etat): string`.

CONTRAINTE: Les fonctions ne doivent pas utiliser `echo`.

10. Activité 6 - Séparer les fichiers

Créez `lib/supervision.php` contenant uniquement les fonctions. Dans `rapport.php` :

```
<?php

declare(strict_types=1);

require_once __DIR__ . "/lib/supervision.php";

$etat = determinerEtat(76);
echo construireMessage("srv-web-01", $etat);
```

QUESTION: Quel problème `DIR` évite-t-il ? Pourquoi utiliser `require_once` ?

11. Activité 7 - Tests simples

Créez `tests.php` :

```
<?php

declare(strict_types=1);

require_once __DIR__ . "/lib/supervision.php";

var_dump(determinerEtat(90) === "CRITIQUE");
var_dump(determinerEtat(70) === "SURVEILLANCE");
var_dump(determinerEtat(69) === "NORMAL");
var_dump(calculerMoyenne([10, 12, 14]) === 12.0);
```

Un résultat `bool(true)` signifie que le cas respecte l'attente.



Ces fonctions de supervision et leurs tests existent en version autonome dans `cours/sandbox-online/05-fonctions.php` : copiez-collez-la sur https://3v4l.org pour

> l'exécuter sans Docker.

12. Diagnostic

Corrigez `diagnostic.php` : les fonctions doivent réellement utiliser leurs paramètres et retourner leurs résultats.

13. Exercice autonome

Créez une bibliothèque `lib/stockage.php` :

- `calculerEspaceUtilise(float $capacite, float $libre): float` ;
- `calculerTauxLibre(float $capacite, float $libre): float` ;
- `determinerEtatStockage(float $tauxLibre): string` ;
- `analyserDisques(array $espacesLibres): array`.

Écrivez au moins six tests couvrant les frontières.

14. Exercices bonus

Facultatifs et gradués, dans le périmètre de la séance (fonctions typées, `return`, décomposition, `require_once` ; ni ternaire, ni `match`, ni classe). Commencez chaque fichier par `declare(strict_types=1);`. Autonomes : testez-les sur https://3v4l.org.

- `[*]` accessible ;
- `[**]` intermédiaire ;
- `[*]` défi.

14.1. Exercice bonus 1 [*] - Un paramètre par défaut

Une fonction peut fournir une valeur par défaut à un paramètre. Écrivez :

```
function calculerCoutAnnuel(float $coutMensuel, int $nombreMois = 12): float
{
    return $coutMensuel * $nombreMois;
}
```

Appelez-la de deux façons et prévoyez la sortie :

```
echo calculerCoutAnnuel(24.90) . PHP_EOL;      // sans préciser les mois
echo calculerCoutAnnuel(24.90, 6) . PHP_EOL;   // sur 6 mois
```

QUESTION: dans quel ordre doit-on placer un paramètre ayant une valeur par défaut ?

14.2. Exercice bonus 2 [**] - Une fonction qui borne une valeur

Les priorités d'incident sont comprises entre 1 et 4. Écrivez une fonction qui **ramène** toute valeur dans cet intervalle :

```
function limiterPriorite(int $priorite): int
{
    // en dessous de 1 -> 1 ; au-dessus de 4 -> 4 ; sinon inchangé
}
```

Testez-la aux bornes : 0, 1, 2, 4, 7. Sortie attendue :

```
0 -> 1
1 -> 1
2 -> 2
4 -> 4
7 -> 4
```

14.3. Exercice bonus 3 [*] - Défi : un mini-outil de test

Jusqu'ici, on vérifiait avec `var_dump(… === …)`, ce qui affiche `bool(true)` / `bool(false)`. Écrivez une fonction d'affichage plus lisible :

```
function verifier(string $libelle, bool $reussi): void
{
    // affiche "[OK]   <libellé>" si $reussi vaut true, "[ÉCHEC] <libellé>" sinon
}
```

Utilisez-la pour tester `determinerEtat()` aux frontières `90`, `89.9`, `70`, `69.9`.

CONTRAINTE: `verifier()` est une fonction d'**affichage** (elle a le droit d'utiliser `echo`), contrairement aux fonctions de **calcul**. Son type de retour est `void` : elle ne retourne rien.

QUESTION: pourquoi est-il logique que `verifier()` retourne `void` alors que `determinerEtat()` retourne une `string` ?

15. Synthèse et Git

1. Une fonction reçoit des … et retourne éventuellement …
2. Une fonction sans valeur retournée est une …

3. `require_once` …
4. Une fonction devrait réaliser …

```
git add 05-fonctions
git commit -m "Décompose la supervision en fonctions testables"
git push
git status
```